書籍管理システム（Global Books Vue3-Django）- スターターキット

これは、Vue.js 3とDjango REST Frameworkを使用して書籍管理システムを構築するためのスターターキットです。 基本的なプロジェクト構成や設定は完了していますが、主要な機能は未実装です。 以下の手順に従って、開発を進めてください。

ZIPの受け取りとVS Codeの準備

以下は、受講者のPCで `C:\work` を作業場所にする例です。

1. **講師からZIPファイルを受け取る**

   配布された `globalbooks_starter_v1.1.0.zip` を自分のPCに保存します。

2. **ZIPを展開する**

   エクスプローラーで `C:\work` フォルダを作成します。ZIPを右クリックして「すべて展開」を選び、展開先を `C:\work` に指定します。

   展開後の作業フォルダは、次の場所です。

   ```
   C:\work\globalbooks_Vue3-Django_starter
   ```

3. **VS Codeで作業フォルダを開く**

   VS Codeの「ファイル」→「フォルダーを開く」から、上記のフォルダを選びます。`backend-django`・`frontend`・`docs`・`README.pdf` が同じ階層に見えるフォルダが正解です。展開時にフォルダが二重になった場合は、その内側を開いてください。

   ```
   globalbooks_Vue3-Django_starter  ← このフォルダを開く
   ├─ backend-django
   ├─ frontend
   ├─ docs
   └─ README.pdf
   ```

4. **ターミナルを開いて場所を確認する**

   VS Codeの「ターミナル」→「新しいターミナル」を開き、PowerShellで次を実行します。

   ```
   cd C:\work\globalbooks_Vue3-Django_starter
   pwd
   dir
   ```

   `pwd` の表示が作業フォルダになり、`dir` で上記のファイル・フォルダが確認できれば準備完了です。別の場所へ展開した場合は、以降の `C:\work\globalbooks_Vue3-Django_starter` を自分の作業フォルダに読み替えます。

チーム開発のルール

この演習は3〜4名のチームで開発します。実装を始める前に、誰がどの部分を担当するか、全員で話し合って決めてください。

設計書を確認し、画面・APIの担当、共通ファイルを変更する担当、結合作業の担当を決めます。同じファイルを複数人が変更する場合は、あらかじめ変更箇所を相談してください。

GitやSubversionなどの共同開発ツールは、基本的に提供しません。講師が用意した共有フォルダを使って、ソースコードを受け渡し・結合します。

作業場所	用途
各自のPCの作業フォルダ	開発・サーバー起動・動作確認を行う
共有フォルダ内の各自用フォルダ	結合するソースコードをアップロードする
共有フォルダ内のチーム代表ソース用フォルダ	結合済みの、チームとしての最終ソースコードを格納する

共有フォルダには、各自用フォルダとチーム代表ソース用フォルダが作成されています。実際の場所・フォルダ名は講師の案内を確認してください。

開発・結合の流れ

1. **各自のPCで開発する**

   チームで担当を決めた後、全員が以下の「セットアップ & 開発手順」で自分の環境を準備します。日々の編集・動作確認はローカルで行います。

2. **各自用の共有フォルダへアップロードする**

   結合する際に、変更したソースコードを自分の共有フォルダへコピーします。元のフォルダ構成を保ち、変更したファイルと内容をチームに伝えてください。

3. **代表ソースへファイルを結合する**

   結合担当が代表ソースと各メンバーの変更をPCへコピーし、ローカルで結合します。同じファイルの変更は、次の「VS Codeで差分を確認して結合する」の手順で見比べ、他の人の変更を消さないようにまとめます。

4. **ローカルで全体を確認し、代表ソースを更新する**

   ローカルで結合したソースで全体が動くか確認し、確認済みの内容を共有フォルダの代表ソース用フォルダへ戻します。各メンバーも最新の代表ソースをローカルへ取り込み、次の開発に進みます。

`node_modules`・`.venv`・`dist`・`__pycache__` は受け渡し不要です。`db.sqlite3` は各自で作成し、ソースコードの結合時には上書きしません。依存パッケージやDB定義を変更した場合は、必要なインストール・マイグレーション手順もチームへ伝えてください。

VS Codeで差分を確認して結合する

VS Code標準のファイル比較機能を使います。Gitや追加の拡張機能は不要です。以下は、同じ `frontend/src/router/index.ts` を2人が変更した場合の例です。

1. **比較するソースをローカルへコピーする**

   共有フォルダから、最新の代表ソース一式とメンバーの変更ファイルを、別々のフォルダへコピーします。元のフォルダ構成を保ちます。次のフォルダ名は一例です。

   ```
   C:\work\結合確認
   ├ メンバーA版\frontend\src\router\index.ts
   └ 代表版\frontend\src\router\index.ts
   ```

2. **2つのフォルダをVS Codeに表示する**

   「ファイル」→「フォルダーを開く」で `C:\work\結合確認` を開きます。左側のエクスプローラーに「メンバーA版」と「代表版」が表示されます。

3. **比較画面を開く**

   エクスプローラーで、メンバーA版の `index.ts` を右クリックし「比較対象の選択」を選びます。続いて、代表版の `index.ts` を右クリックし「選択項目と比較」を選びます。

左：メンバーA版（参照）	右：代表版（結合）
メンバーが実装した内容を確認	必要な変更を反映して保存

   違う行が色付きで表示されます。色は正解・不正解を表すものではありません。横並びにならない場合はウィンドウを広げ、比較画面右上の「…」で「インライン ビュー」を解除します。

4. **必要な変更を代表版へ反映する**

   差分を1か所ずつ確認し、右側の代表版へ必要な行をコピー・編集します。代表版にある他のメンバーの変更も残してください。同じ行を双方が変更している場合は担当者と相談します。比較画面を開いただけでは、自動で結合されません。

5. **保存・動作確認・共有を行う**

   代表版を `Ctrl+S` で保存します。VS Codeで代表版のプロジェクトフォルダを開き直し、サーバーを起動して関係する画面・APIを確認します。確認できたソースを共有フォルダの代表ソース用フォルダへ戻します。

機能要件（実装目標）

- 書籍一覧表示（画像付き）
- 書籍詳細表示
- 新規書籍登録
- 書籍編集
- 書籍削除

技術スタック

- **Frontend**: Vue.js 3, Vite, Bootstrap 5
- **Backend**: Django 5, Django REST Framework
- **Database**: SQLite3

セットアップ & 開発手順

前提条件

- Python 3.10+（演習環境は3.13）
- Node.js 20.19以上の20系、または22.12以上（今回のVite 7に合わせて変更）
- SQLite3コマンドラインツール（`sqlite3` コマンドを実行できること）

追記： 演習で使うPythonパッケージを、最初に1回インストールしてください。

```
python -m pip install "Django>=5.2.10,<5.3" "djangorestframework==3.16.1" "django-cors-headers==4.9.0"
```

1. バックエンド (Django) のセットアップ

バックエンド用のDjangoディレクトリと、DB作成用SQLだけ準備してあります。 以下の手順に従いセットアップを行ってください。

手順5・6では、ファイルの編集・保存が必要です。コマンドの実行だけでは書籍テーブルは作成されません。

1. **ディレクトリ移動**

   このREADMEのあるフォルダから実行します。

```
cd backend-django
```

2. **Djangoプロジェクトの作成**

   Djangoプロジェクトを作成します。

```
django-admin startproject config .
```

3. **アプリケーションの作成**

   書籍管理用のアプリ `books` を作成します。

```
python manage.py startapp books
```

4. **ディレクトリ構成 (Backend)**

   以下のディレクトリ構成になっていることを確認してください。 （実行結果は一部省略して表示しています）

```
tree /f ./
```

```
backend-django
│  initial_data.sql
│  manage.py
│
├─books
│  │  models.py
│  │  views.py
│  │
│  └─migrations
│
└─config
   │  settings.py
   └─ urls.py
```

5. **設定の追加**

`config/settings.py` を開き、次の設定を追加して `Ctrl+S` で保存します。

- 既存の `INSTALLED_APPS = [` から対応する `]` の内側に、次の3つを追加します。最初からあるアプリは残します。

```
'rest_framework',  # 追加: REST API用
'corsheaders',     # 追加: CORS用
'books',           # 新規: 書籍管理アプリ
```

- `MIDDLEWARE` にCORSの設定を先頭に追加

```
'corsheaders.middleware.CorsMiddleware',
```

- `CORS_ALLOWED_ORIGINS` を定義して、オリジンサーバを追加

```
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5173',
    'http://localhost:5174',
    'http://localhost:5175',
]
```

6. **モデルの作成**

VS Codeのエクスプローラーで `backend-django` → `books` → `models.py` を開きます。`startapp books` が作るファイルは雛形で、`Book` モデルは自動生成されません。

この時点の `books/models.py` の内容を、次のコード全文に置き換えて `Ctrl+S` で保存してください。 これはファイルに記述するPythonコードで、ターミナルへ入力するコマンドではありません。

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=255)
    author = models.CharField(max_length=255)
    price = models.IntegerField()
    publisher = models.CharField(max_length=255, null=True, blank=True)
    image_path = models.CharField(max_length=255, null=True, blank=True)
    # 追加機能専用（いいね・いいね数順）。基本機能では使わず、列だけ先に用意する。
    like_count = models.PositiveIntegerField(default=0)

    class Meta:
        db_table = 'books'

    def __str__(self):
        return self.title
```

基本設計書の「2.1 テーブル定義: books」に対応するモデルです。インデントも含めてコピーしてください。

設定	意味
id	Djangoが自動生成するため、コードへの追加は不要
null=True, blank=True	出版社・画像パスは未入力を許可
db_table = 'books'	初期SQLと同じテーブル名に指定
like_count	追加機能専用。基本機能では使わず、初期値0で列だけ先に用意

7. データベースのマイグレーション

手順5・6の編集内容を保存してから、`backend-django` のPowerShellで1行ずつ実行します。エラーが出た場合は、修正してから次へ進みます。

```
python manage.py makemigrations books
python manage.py migrate
python manage.py showmigrations books
```

初回は `books/migrations/0001_initial.py` が作成され、`migrate` で `Applying books.0001_initial... OK` と表示されます。最後に次の表示を確認してから、初期データの投入へ進みます。

```
books
 [X] 0001_initial
```

表示された内容	確認・対応
No installed app with label 'books'	手順5に戻り、INSTALLED_APPSの内側にbooksを追加して保存
初回なのにNo changes detected／(no migrations)	手順6に戻り、books/models.pyのBookモデルを実装して保存
[ ] 0001_initial	まだ未適用。migrateを実行して[X]になることを確認

すでに `[X] 0001_initial` となっていてモデルに変更がなければ、再実行時の `No changes detected` は正常です。`admin`・`auth` などのマイグレーションだけでは、書籍テーブルは作成されません。

8. **初期データの投入（SQL）**

   `backend-django` フォルダに `initial_data.sql` が用意されています。 このSQLはデータの登録用です。テーブルの作成は手順7で行います。

   PowerShell（PS ...>）で入力：

   ```
   sqlite3 db.sqlite3
   ```

   SQLite（sqlite>）に切り替わったら、まず入力：

   ```
   .tables
   ```

   一覧に `books` があることを確認します。なければ `.quit` で終了し、手順5〜7を確認してください。既存の `db.sqlite3` を削除する必要はありません。

   booksが存在し、まだ書籍を登録していない場合だけ、SQLiteで入力：

   ```
   .read initial_data.sql
   SELECT COUNT(*) FROM books;
   .quit
   ```

   書籍10件が表示され、件数が `10` になれば完了です。同じ初期データを再投入しないでください。

   `.read`・`.tables`・`.quit` は、行頭のドットも含めて入力します。ドットなしの `read` や `quit` を入力して `..>` になった場合は、`Ctrl+C` で中断します。`sqlite>` に戻ったら `.quit` で終了します（`PS ...>` に戻った場合は終了済みです）。

9. **サーバー起動**

   ```
   python manage.py runserver
   ```

   標準ポート8000で起動します。

2. フロントエンド (Vue) のセットアップ

フロントエンドは、必要な画面やコンポーネントのファイル（枠組み）のみ用意されています。

1. **統合ターミナルを開いてインストールする**

   Djangoのサーバーは起動したままにします。VS Code左側のエクスプローラーで `frontend` フォルダを右クリックし、「統合ターミナルで開く」を選びます。`frontend` を作業場所とする新しいターミナルが開くので、次を実行します。

   ```
   npm install
   ```

2. **NPM スクリプトから開発サーバーを起動する**

   インストール完了後、`frontend/package.json` を開き、エクスプローラー下部の「NPM スクリプト」を表示します。`frontend` の `package.json` にある `dev`（実行内容は `vite --port 5173 --strictPort`）にマウスを合わせ、右側の再生ボタンをクリックします。

   「NPM スクリプト」→ `frontend` の `package.json` → `dev` の再生ボタン → Vite起動

   「NPM スクリプト」が見えない場合は、`Ctrl+Shift+P` で「表示: ビューを開く」を選び、「NPM スクリプト」を開きます。起動したターミナルは、そのまま開いておきます。

3. **ブラウザで確認**

   アクセス: http://localhost:5173

   画面に表示される指示（コメント）に従って、実装を進めてください。

補足：コマンドで操作する場合

上記の画面操作の代わりに、別のターミナルで次を実行しても同じ準備・起動ができます。すでにインストール・起動済みの場合は、重ねて実行する必要はありません。

```
cd C:\work\globalbooks_Vue3-Django_starter
cd frontend
npm install
npm run dev
```

補足

・ APIの仕様や画面設計については、別途配布される設計書を確認してください。

追記：今回の追加機能（任意）

基本機能を完成させた後、次の3つに任意で取り組めます。

・ 並び替えボタンの追加

・ 書籍ごとのいいね

・ 画像ファイル添付

いいね用の列は手順6で用意済みです。APIの実装時には、`BookSerializer` の `class Meta` に `read_only_fields = ['like_count']` を指定し、通常の登録・編集で回数を上書きしないようにします。

画像ファイル添付に取り組む場合だけ、Pillowを追加でインストールしてください。

```
python -m pip install "Pillow>=11.1,<13"
```

画面例は同梱の基本設計書の第7章「追加機能」を確認してください。